

Software Requirements Specification (SRS)

Hostel Management System

Project: Hostel Management System

Team: Team 5

Version: 1.0

Date: 04-09-2026

Status: Draft for Review

Team Members

Name	SRN
Arjav Chandranath	PES2UG24CS080
Arjun Mohan Gowda	PES2UG24CS082
Armaan Ali	PES2UG24CS083
Arnav Sharath Hanchanur	PES2UG24CS084

Revision History

Version	Date	Author	Change Summary	Approval
0.1	01-09-2026	Team 5	Initial draft structure and scope	Pending
1.0	04-09-2026	Team 5	Complete SRS with FRs, NFRs, security requirements, UML diagrams and RTM	Pending

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			

Table of Contents

- 1. Introduction
- 2. Overall Description
- 3. External Interface Requirements
- 4. System Features (Detailed)
- 5. Non-Functional Requirements (Detailed)
- 6. Quality Attributes & Acceptance Tests
- 7. System Models and Diagrams
- 8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This document is the Software Requirements Specification (SRS) for the Hostel Management System (HMS), a web application for managing student hostel room allocation, student records, fee collection, and maintenance requests. It defines the functional and non-functional requirements, interfaces, and verification criteria intended for the development team, course evaluators, and future maintainers.

1.2 Scope

The system covers online student registration, room allocation and reallocation across hostel blocks, student profile and record management, hostel fee configuration and online payment, maintenance/complaint tracking, and administrative reporting. It excludes physical security systems (biometric access, CCTV), mess/canteen management, and integration with the university's central ERP beyond basic student-ID verification, which are treated as out of scope for this academic project.

1.3 Audience

Developers, QA testers, the Course Coordinator/Evaluators, the Hostel Warden and administrative staff (as end users), and future student teams maintaining the system.

1.4 Definitions, Acronyms and Abbreviations

- HMS - Hostel Management System
- RBAC - Role-Based Access Control
- ORM - Object-Relational Mapping
- CRUD - Create, Read, Update, Delete
- OTP - One-Time Password
- API - Application Programming Interface
- PCI-DSS - Payment Card Industry Data Security Standard
- TLS - Transport Layer Security
- RTM - Requirements Traceability Matrix
- WCAG - Web Content Accessibility Guidelines

2. Overall Description

2.1 Product Perspective

HMS is a standalone, self-contained web application built with a Python (Django) backend, a relational database (PostgreSQL), and a server-rendered frontend (Django templates with Bootstrap, optionally augmented with JS for interactivity). It is not part of a larger product family; where the university's central student database exists, HMS integrates only at the level of validating a student's roll number/ID during registration.

2.2 Major Product Functions

- User registration, authentication and role-based access (Student, Warden/Admin)
- Room allocation, reallocation and hostel block/room inventory management
- Student record management (personal details, guardian/emergency contact, room history)
- Hostel fee structure configuration, online fee payment and receipt generation
- Maintenance/complaint submission and resolution tracking
- Administrative dashboards and reports (occupancy, vacancy, fee collection, defaulters)

2.3 User Roles and Characteristics

- Student: basic computer literacy; uses the system primarily during admission/allocation windows and for fee payment and complaint tracking; expects a simple, mobile-friendly interface.
- Warden / Admin: manages allocations, fee structures and reports; needs efficient bulk operations (e.g., allocating an entire incoming batch) and clear audit trails.
- Technician (optional sub-role of Admin): views and updates status of maintenance complaints assigned to them.

2.4 Operating Environment

Server-side: Linux-hosted Django application behind a WSGI/ASGI server (e.g., Gunicorn) and reverse proxy (Nginx), PostgreSQL database. Client-side: any modern desktop or mobile browser (Chrome, Firefox, Edge, Safari) with a minimum viewport of 360px width supported via responsive design.

2.5 Constraints

- Must be deliverable and demonstrable within the academic semester timeline.
- Payment processing must use a sandbox/test mode of a PCI-DSS compliant gateway (e.g., Razorpay/Stripe test mode) - no real cardholder data is handled directly by HMS.
- Database design must support the course's DBMS integration expectations (relational storage for transactional data).
- Team of four students, within a single-semester project timeline.

3. External Interface Requirements

3.1 User Interfaces

A responsive web UI with distinct dashboards per role: a Student Dashboard (room details, fee status, complaints) and an Admin/Warden Dashboard (allocation console, fee configuration, reports). Accessibility: semantic HTML, keyboard navigation, and sufficient colour contrast targeting WCAG 2.1 AA where practical.

3.2 Hardware Interfaces

None required beyond a standard client device (desktop, laptop, or smartphone) capable of running a modern web browser; the server runs on standard commodity/cloud infrastructure (e.g., a VM or PaaS instance) with no specialised hardware dependency.

3.3 Software Interfaces

- Django ORM and PostgreSQL for persistent storage of students, rooms, allocations, fee transactions and complaints.
- Payment Gateway API (test/sandbox mode) for online fee payment and webhook-based payment confirmation.
- Email/SMS Gateway (ex: SMTP, or a transactional email API) for OTP delivery, allocation notifications and fee reminders.
- Optional REST API layer (Django REST Framework) exposing endpoints for allocation, fee and complaint data, enabling future mobile-app or reporting-tool integration.

3.4 Communications

- TLS 1.2+ enforced for all client-server traffic (HTTPS only; HTTP requests redirected).
- Session-based authentication with CSRF protection for the web app; token-based (JWT) authentication if the REST API is exposed externally.

4. System Features (Detailed)

Synopsis:

This SRS defines 20 Functional Requirements, 6 Non-Functional Requirements, 3 security objectives and 5 Security Requirements in total.

Each requirement below includes acceptance criteria and a reference test case. IDs follow HMS-F-###. A total of 18 functional requirements is specified across six feature groups.

4.1 Authentication, Profile & Access Control

Description: Register/authenticate users, maintain profiles, and enforce role-based access across Student and Warden/Admin roles.

Req ID	Requirement	Type	Priority	Source/Stakeholder	Acceptance Criteria / Test Ref	Comments/Dependencies
HMS-F-001	The system shall allow a user to register and log in using institute email/roll number, with role assigned as Student, Warden or Admin.	Functional	High	All users	AC-F-001: Valid credentials create an authenticated session with correct role. Test: TC-Auth-01	Depends on institute ID verification
HMS-F-002	The system shall allow a student to update their profile (contact number, address, emergency contact) after login.	Functional	Medium	Student	AC-F-002: Updated fields persist and are reflected on next login. Test: TC-Auth-02	-
HMS-F-003	The system shall support password reset via a one-time password (OTP) sent to the registered email.	Functional	Medium	All users	AC-F-003: OTP expires in 10 minutes; valid OTP allows password reset. Test: TC-Auth-03	Requires email gateway
HMS-F-004	The system shall automatically terminate an inactive session after 20 minutes and require re-authentication.	Functional	High	Security	AC-F-004: Session expires and protected pages redirect to login after timeout. Test: TC-Auth-04	-
HMS-F-005	The system shall enforce role-based access control (RBAC) so that Students cannot access Admin/Warden views and vice versa.	Functional	High	Security	AC-F-005: Unauthorized role access returns HTTP 403 and is logged. Test: TC-Auth-05	Enforced at view and API layer

4.2 Room Allocation & Inventory Management

Description: Manage hostel block/room inventory and allocate, reallocate or transfer students between rooms while preserving capacity and policy constraints.

Req ID	Requirement	Type	Priority	Source/Stakeholder	Acceptance Criteria / Test Ref	Comments/Dependencies
HMS-F-010	The system shall allow a logged-in student to view their current room allocation and roommate details.	Functional	High	Student	AC-F-010: Allocated room number, block and roommates are displayed correctly. Test: TC-Alloc-01	Requires allocation to exist
HMS-F-011	The system shall allow the Admin/Warden to create, view, update and deactivate student records.	Functional	High	Admin	AC-F-011: CRUD operations reflect immediately in the student list. Test: TC-Alloc-02	-
HMS-F-012	The system shall allow the Admin/Warden to allocate or reallocate a room to a student based on availability, gender policy and stated preferences.	Functional	High	Admin/Operations	AC-F-012: Allocation only succeeds for rooms with available capacity; occupancy updates atomically. Test: TC-Alloc-03	Core allocation engine
HMS-F-013	The system shall allow a student to submit a room change request, and the Admin/Warden to approve or reject it.	Functional	Medium	Student/Admin	AC-F-013: Request status transitions (Pending → Approved/Rejected) are logged with timestamp. Test: TC-Alloc-04	Approval triggers reallocation (HMS-F-012)
HMS-F-014	The system shall allow the Admin to manage hostel blocks, floors, rooms and their capacity/type (single/shared).	Functional	High	Admin	AC-F-014: Room inventory changes are reflected in real-time availability counts. Test: TC-Alloc-05	-
HMS-F-015	The system shall prevent allocation beyond a room's configured capacity, rejecting over-allocation attempts with a clear error.	Functional	High	Data Integrity	AC-F-015: Allocation attempt on a full room is blocked and logged. Test: TC-Alloc-06	Database-level constraint + application check

4.3 Hostel Fee Management

Description: Configure fee structures, process online payments, and reconcile collections against dues.

Req ID	Requirement	Type	Priority	Source/Stakeholder	Acceptance Criteria / Test Ref	Comments/Dependencies
HMS-F-020	The system shall allow a student to view an itemised fee statement including dues, paid amount and due date.	Functional	High	Student	AC-F-020: Statement totals match the configured fee structure minus payments made. Test: TC-Fee-01	-
HMS-F-021	The system shall allow a student to pay hostel fees online through an integrated (sandbox) payment gateway.	Functional	High	Student/Finance	AC-F-021: Successful payment updates dues and triggers a confirmation notification. Test: TC-Fee-02	Requires payment gateway API
HMS-F-022	The system shall allow the Admin to configure the fee structure per academic term and room type.	Functional	Medium	Admin/Finance	AC-F-022: New fee structure applies to subsequent statements without affecting already-paid records. Test: TC-Fee-03	-
HMS-F-023	The system shall generate and allow download of a PDF payment receipt after each successful transaction.	Functional	Medium	Student	AC-F-023: Receipt contains transaction ID, amount, date and student details. Test: TC-Fee-04	-
HMS-F-024	The system shall allow the Admin to reconcile payments against the gateway ledger and flag fee defaulters.	Functional	High	Admin/Finance	AC-F-024: Defaulter list matches students with dues past the due date. Test: TC-Fee-05	Depends on gateway webhook (HMS-F-021)

4.4 Maintenance & Complaint Tracking

Description: Allow students to raise facility issues and staff to track resolution to closure.

Req ID	Requirement	Type	Priority	Source/Stakeholder	Acceptance Criteria / Test Ref	Comments/Dependencies
HMS-F-030	The system shall allow a student to raise a maintenance complaint with category, description and optional photo attachment.	Functional	Medium	Student	AC-F-030: Complaint is created with a unique ticket ID and "Open" status. Test: TC-Maint-01	-
HMS-F-031	The system shall allow the Admin/technician to update complaint status (Open → In Progress → Resolved) and notify the student.	Functional	Medium	Admin/Technician	AC-F-031: Status changes are timestamped and visible to the student. Test: TC-Maint-02	-

4.5 Administrative Reporting

Description: Provide occupancy, vacancy, fee-collection and defaulter reports to support hostel management decisions.

Req ID	Requirement	Type	Priority	Source/Stakeholder	Acceptance Criteria / Test Ref	Comments/Dependencies
HMS-F-040	The system shall allow the Admin to generate occupancy, vacancy and fee-collection reports for a selected date range.	Functional	Medium	Admin/Management	AC-F-040: Report figures match underlying allocation and payment records. Test: TC-Rep-01	-
HMS-F-041	The system shall allow reports to be exported in CSV and PDF formats.	Functional	Low	Admin/Management	AC-F-041: Exported file opens correctly and matches on-screen data. Test: TC-Rep-02	Depends on HMS-F-040

5. Non-Functional Requirements (Detailed)

NFRs below are measurable and tied to test plans. IDs follow HMS-NF-####.

Req ID	Requirement	Category	Priority	Acceptance Criteria / Measurement
HMS-NF-001	95% of page loads and form submissions shall complete within 2 seconds under normal load (up to 200 concurrent users).	Performance	High	Load test (e.g., Locust) shows p95 $\leq$ 2s. Test: TC-Perf-01
HMS-NF-002	The system shall be available 99.5% of the time during the academic semester, excluding scheduled maintenance windows.	Reliability	High	Uptime monitoring report $\geq$ 99.5% per semester. Test: Ops log review
HMS-NF-003	All student personal data and payment records shall be protected in compliance with standard data-protection practice; PANs/card data are never stored on HMS servers.	Security/Compliance	High	Security review checklist pass; DB audit shows no raw card data. Test: TC-Sec-01
HMS-NF-004	The system shall retain fee-transaction and allocation audit logs for a minimum of 3 years.	Audit/Data Retention	High	Archived logs retrievable on request. Test: TC-Ops-01
HMS-NF-005	The UI shall be responsive across desktop and mobile viewports ($\geq$ 360px width) and follow WCAG 2.1 AA guidance for colour contrast and keyboard navigation where practical.	Usability/Accessibility	Medium	Manual accessibility audit pass. Test: TC-UX-01
HMS-NF-006	The system architecture shall support at least 5,000 student records and 2,000 rooms without redesign, and shall be modular (separate Django apps per feature).	Scalability/Maintainability	Medium	Load/scale test with seeded data of target size completes within performance targets. Test: TC-Scale-01

5.1 Security

5.1.1 Security Objectives

- **Confidentiality & Access Control:** Ensure only authenticated, correctly-authorised users (Student vs. Warden/Admin) can view or modify data appropriate to their role.
- **Data Protection:** Protect student personal information and payment-related data both in transit (TLS) and at rest (hashing/no raw card storage).
- **Integrity & Auditability:** Maintain a verifiable audit trail for room allocation changes and fee transactions so that actions can be traced and disputes resolved.

5.1.2 Security Requirements

Req ID	Requirement	Type	Priority	Acceptance Criteria / Test Ref
HMS-SR-001	TLS 1.2+ shall be mandatory for all client-server network connections; plain HTTP requests are redirected to HTTPS.	Security	High	SSL Labs / server config scan confirms TLS 1.2+ only. Test: TC-Sec-02
HMS-SR-002	User passwords shall be stored using a salted, industry-standard hash (Django's PBKDF2/Argon2), never in plaintext.	Security	High	DB inspection confirms hashed storage; no plaintext password recoverable. Test: TC-Sec-03
HMS-SR-003	Role-based access control shall be enforced at both the UI (template/view) and API layer, not the UI alone.	Security	High	Direct API calls with a student token to Admin endpoints return HTTP 403. Test: TC-Sec-04
HMS-SR-004	All administrative actions (allocation changes, fee-structure edits, complaint status changes) shall be recorded in an immutable audit log with user, timestamp and action.	Security	High	Audit log entries verified for a sample set of admin actions. Test: TC-Sec-05
HMS-SR-005	Online payments shall be processed exclusively through a PCI-DSS compliant gateway (sandbox for this project); HMS shall never store raw card/CVV data.	Security	High	Code/DB review confirms no card-data fields exist in the HMS schema. Test: TC-Sec-06

6. Quality Attributes & Acceptance Tests

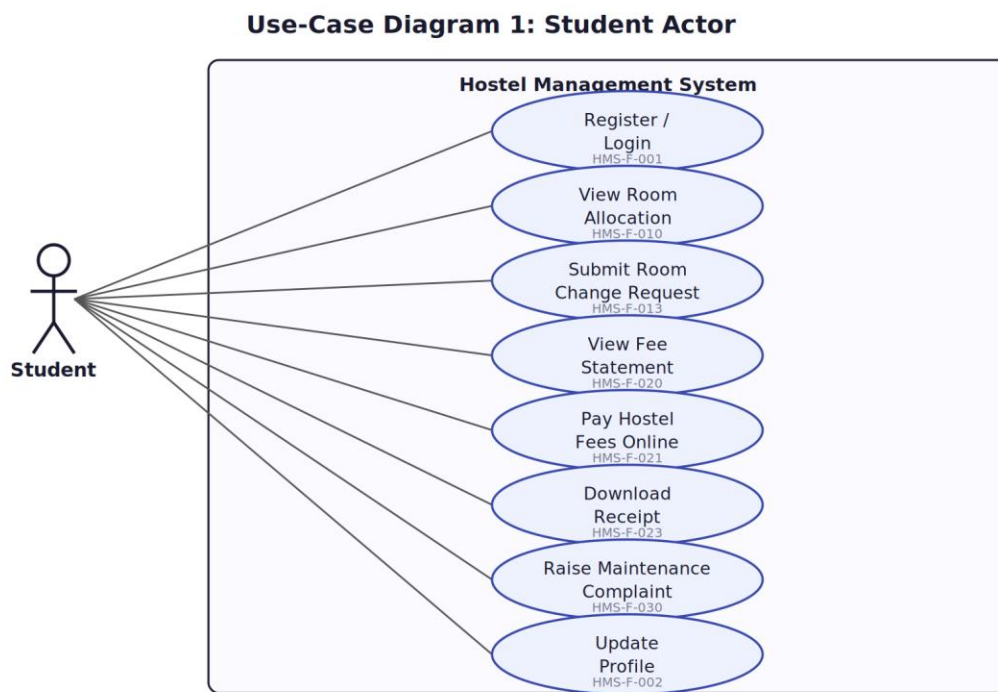
Exit criteria for acceptance: all High-priority functional requirements implemented and verified, no critical NFR failures, and the RTM shows all mapped test cases passed.

Acceptance test suites: Authentication & Access Control, Room Allocation, Fee Payment, Maintenance Tracking, Reporting, Performance, Security, and Accessibility.

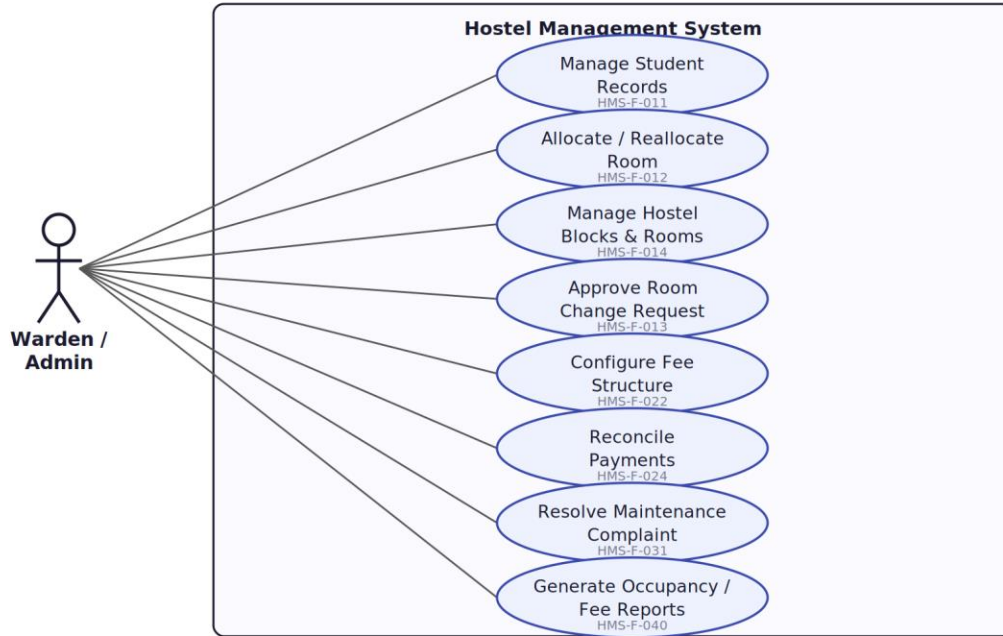
7. System Models and Diagrams

7.1 UML Use-Case Diagrams

Two use-case diagrams are provided below, covering the “Student” actor and the Warden/Admin actor respectively. Together they cover all 18 functional requirements defined in Section 4.



Use-Case Diagram 2: Warden / Admin Actor



8. Requirements Traceability Matrix (RTM)

Status legend - N: Not tested, P: Partially tested, A: All test cases passed.

Req ID	Requirement (short)	Section Ref / Design Spec	Module	Test Case(s)	Status (N/P/A)	Comments
HMS-F-001	Register / Login (RBAC)	4.1 / DS-Auth-01	AuthModule	TC-Auth-01	N	-
HMS-F-005	Enforce RBAC	4.1 / DS-Auth-02	AuthModule	TC-Auth-05	N	-
HMS-F-012	Allocate / reallocate room	4.2 / DS-Alloc-01	AllocationModule	TC-Alloc-03	N	Core module
HMS-F-015	Prevent over-allocation	4.2 / DS-Alloc-02	AllocationModule	TC-Alloc-06	N	DB constraint
HMS-F-021	Online fee payment	4.3 / DS-Fee-01	FeeModule	TC-Fee-02	N	Gateway sandbox
HMS-F-024	Reconcile payments / defaulters	4.3 / DS-Fee-02	FeeModule	TC-Fee-05	N	-
HMS-F-030	Raise maintenance complaint	4.4 / DS-Maint-01	ComplaintModule	TC-Maint-01	N	-
HMS-F-040	Generate reports	4.5 / DS-Rep-01	ReportingModule	TC-Rep-01	N	-
HMS-NF-001	Response time target	5 / DS-Perf-01	WebUI / All Modules	TC-Perf-01	N	-
HMS-SR-002	Salted password hashing	5.1.2 / DS-Sec-01	AuthModule	TC-Sec-03	N	-